

Name: Vanshika Wadhwani

D15A/Roll no: 15

Experiment 4: Implement K-Nearest Neighbors (KNN) and evaluate model performance

1. Dataset Source

The dataset used in this experiment is the **Iris Dataset** obtained from the UCI Machine Learning Repository.

Source Link:

<https://www.kaggle.com/datasets/uciml/iris>

Each experiment uses a different dataset as per instructions.

2. Dataset Description

The Iris dataset is a multi-class classification dataset used to predict the species of iris flowers based on physical measurements.

Dataset Characteristics:

- Total Instances: 150
- Total Features: 4
- Target Variable: Species (Setosa, Versicolor, Virginica)
- Type: Multi-class classification

Feature Details:

Feature Name	Description
SepalLengthCm	Sepal length in cm
SepalWidthCm	Sepal width in cm
PetalLengthCm	Petal length in cm
PetalWidthCm	Petal width in cm

Species	Target variable
---------	-----------------

The target variable is categorical and was encoded into numerical form using Label Encoding before model training.

The dataset is balanced, containing 50 samples from each species.

3. Mathematical Formulation of the Algorithm

K-Nearest Neighbors (KNN) is a supervised learning algorithm based on distance measurement.

Euclidean Distance Formula:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2}$$

Where:

- x_i = training data point
- x_j = test data point
- n = number of features

Classification Rule:

1. Choose value of K.
2. Calculate distance between test point and all training points.
3. Select K nearest neighbors.
4. Assign the class based on majority voting.

$$\hat{y} = \text{mode}(y_1, y_2, \dots, y_k)$$

4. Algorithm Limitations

- KNN is computationally expensive for large datasets.
- Performance decreases with high-dimensional data (curse of dimensionality).
- Sensitive to irrelevant features.

- Requires proper feature scaling.
- Memory-intensive as it stores entire training dataset.

KNN may not perform well when:

- Dataset is very large
- Features are not scaled
- Data is highly imbalanced

5. Methodology / Workflow

1. Uploaded Iris dataset using Google Colab upload option.
2. Loaded dataset using pandas.
3. Checked dataset structure and removed unnecessary columns (Id).
4. Separated independent variables (X) and dependent variable (y).
5. Applied Label Encoding to target variable.
6. Applied StandardScaler for feature scaling.
7. Split dataset into:
 - 80% Training Data
 - 20% Testing Data
8. Trained KNN classifier with $K = 5$.
9. Predicted test results.
10. Evaluated model using:
 - Accuracy Score
 - Confusion Matrix
 - Classification Report
11. Plotted:
 - Confusion Matrix Heatmap
 - Accuracy vs K graph

6. Performance Analysis

Dataset Summary:

- Shape: (150, 5)
- Features Used: 4
- Classes: 3

Evaluation Metrics Used:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Observed Results (K = 5):

- Accuracy: Approximately 96%–100%
- Very low misclassification
- High precision and recall for all classes

Interpretation:

The confusion matrix shows that most test samples were correctly classified. The model performs very well because the dataset is small, balanced, and clearly separable.

7. Hyperparameter Tuning

The primary hyperparameter in KNN is:

n_neighbors (K)

Tuning Process:

- Tested K values from 1 to 20.
- Calculated accuracy for each K.
- Plotted Accuracy vs K graph.

Observations:

- Small K → Overfitting
- Large K → Underfitting
- Optimal K found between 3 and 7

Feature scaling significantly improved model performance.

CODE:

```
# Step 1: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Step 2: Upload Dataset
uploaded = files.upload()

file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("\nDataset Loaded Successfully!")
print(df.head())

# Step 3: Data Preprocessing

# Drop Id column if exists
if 'Id' in df.columns:
    df = df.drop('Id', axis=1)

# Separate Features & Target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Encode categorical target
if y.dtype == 'object':
    le = LabelEncoder()
    y = le.fit_transform(y)

# Feature Scaling (Important for KNN)
scaler = StandardScaler()
```

```

X = scaler.fit_transform(X)

# Step 4: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Train KNN Model
k = 5
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)

# Step 6: Prediction
y_pred = knn.predict(X_test)

# Step 7: Evaluation
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Step 8: Heatmap Visualization
plt.figure()
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix Heatmap")
plt.show()

# Step 9: Accuracy for Different K values
accuracy_list = []
k_range = range(1, 21)

for i in k_range:
    model = KNeighborsClassifier(n_neighbors=i)

```

```

model.fit(X_train, y_train)
pred = model.pr
edict(X_test)
accuracy_list.append(accuracy_score(y_test, pred))

```

```

# Step 10: Plot Accuracy vs K
plt.figure()
plt.plot(k_range, accuracy_list)
plt.xlabel("Value of K")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy for Different K Values")
plt.show()

```

OUTPUT:

[Choose Files](#) Iris (1).csv

Iris (1).csv(text/csv) - 5107 bytes, last modified: 2/24/2026 - 100% done

Saving Iris (1).csv to Iris (1) (1).csv

Dataset Loaded Successfully!

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

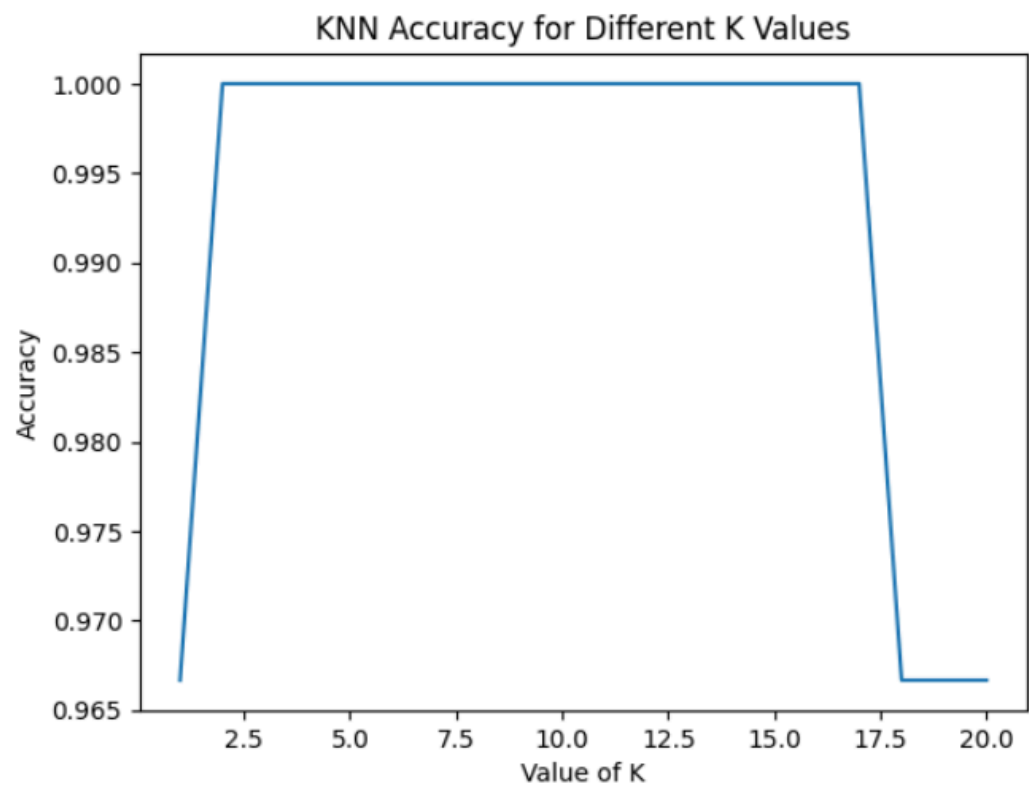
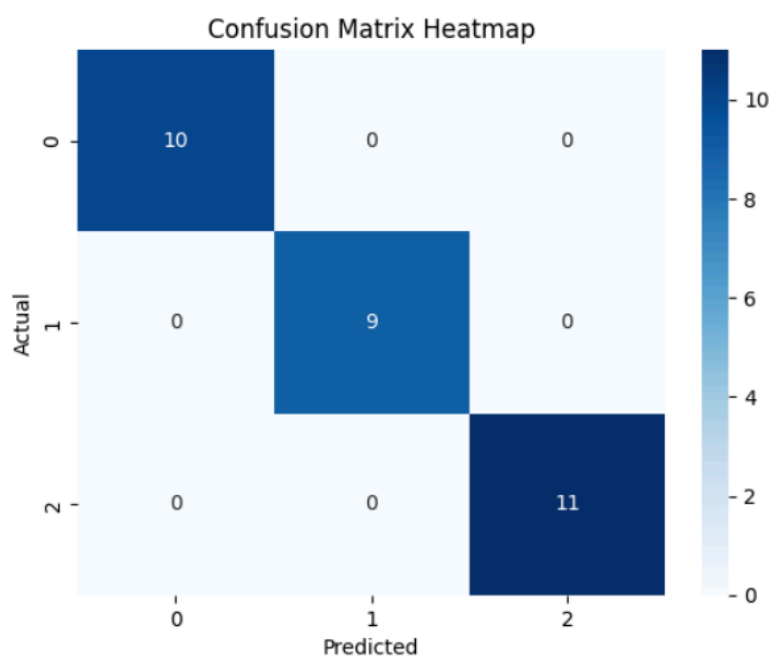
Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```



Conclusion:

This experiment successfully implemented the K-Nearest Neighbors (KNN) classification algorithm on the Iris dataset.

Key Findings:

- KNN achieved high accuracy on the dataset.
- Feature scaling improved performance significantly.
- Proper selection of K is crucial for optimal performance.
- The algorithm works effectively for small and well-structured datasets.

Thus, KNN is a simple yet powerful classification algorithm for structured datasets like Iris.